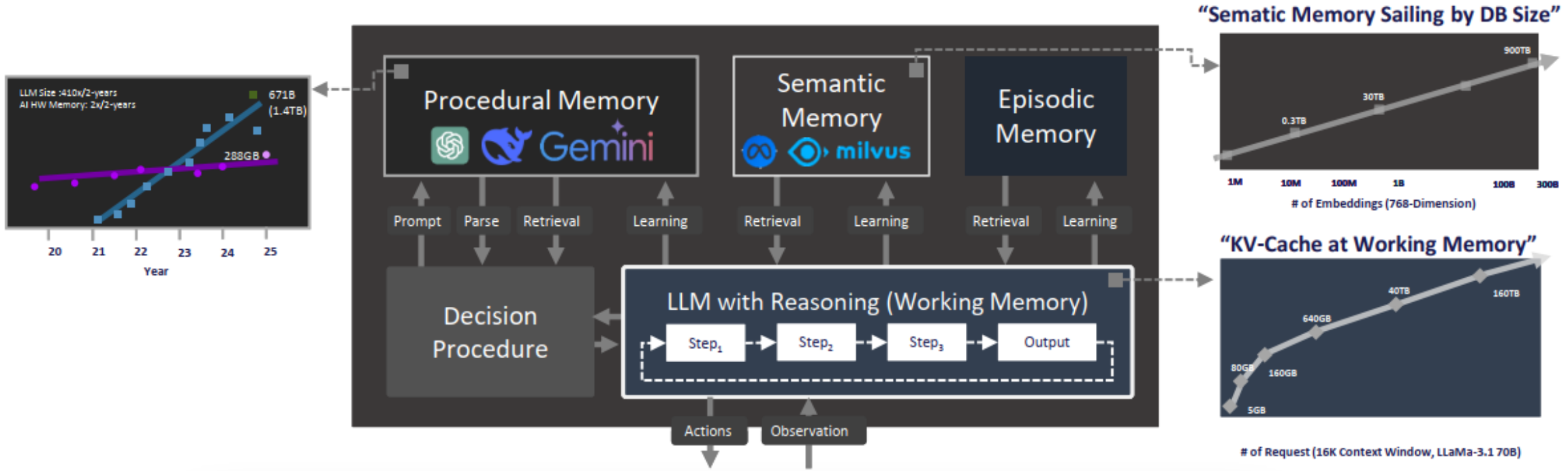
AGENT WITH AN INTERCONNECT SOLUTION

Yiwei Yang, Zettai LLC

LLM BASED AGENTIC AI: MEMORY VIEW



Agentic AI based on LLM with KV cache requires significant memory capacity and bandwidth relying on external databases

- (Working) A space where agent temporarily stores and processes information current tasks or reasoning.
- (Procedural) Implicit knowledge stored in LLM weights.
- (Semantic) Agent's knowledge about the world and itself.
- (Episodic) Experiences from earlier decisions(e.g. training input-output pairs, history event flows, and game trajectories)

Scalability Problem

BACKGROUND AND MOTIVATION

The challenges to resolve the memory capacity and performance of Next-Gen Memory-Intensive App

1. The HBM availability, supply stability, and high TCO

Those factors are slowing down the AI server development and grow.In this new era, we should explore how to use fewer HBM. Instead, increase host memory capacity and performance. This will keep scalability, supply availability and lower TCO.

2. Memory subsystem scalability and lower the TCO requirement

The diversity of Memory-Intensive App Server architectures depends ontheir use cases and workloads . How to avoid the under-provisioning,over-provisioning of the memory subsystem and stranded memory are the challenges to Next-Gen Memory-Intensive App Server architectures today

3. The need for bigger memory capacity and higher performance per CPU-socket

Scaling a host memory capacity and performance per CPU socket is becoming increasingly challenging each year, yet DLRM, IMDB, AI and HPC workloads continue to drive the demand of CPU sockets toward bigger capacity and higher performance

4. The agent demand for CPU.

Tool processing on the CPU can dominate end-to-end latency, accounting for up to 90.6% of the total latency. Meanwhile, CPU dynamic energy can account for up to 44% of the total system dynamic energy.

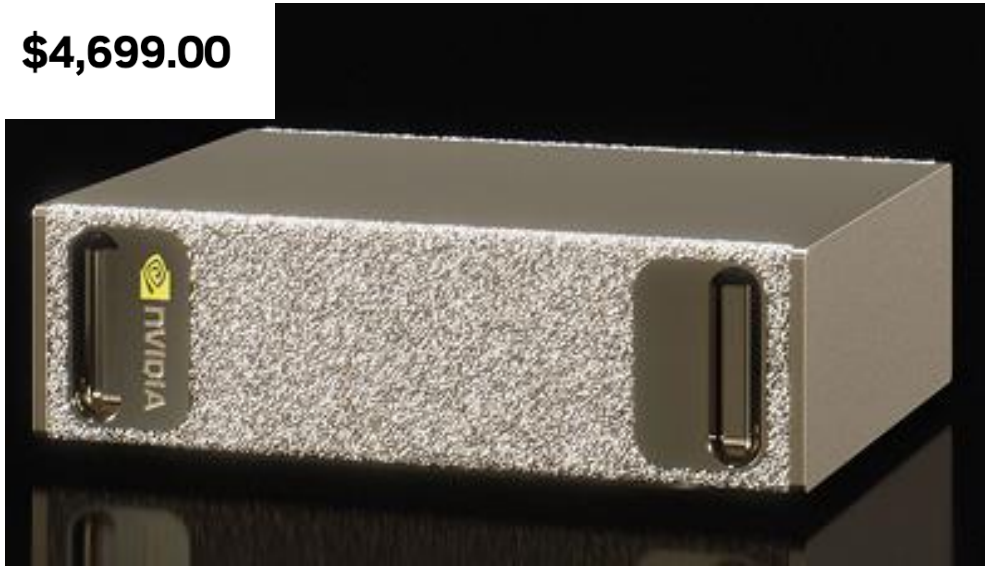
---

* Host Memory (CPU-manage memory > CPU near memory, far memory, pooled memory) JBOM

TARGET MARKET

$7,349.00 or $612.41/mo. for 12 mo.‡

$4,699.00



Price

USD $3,000

$45,000–$50,000

FLAGSHIP **ZettEngine**

Our flagship AI acceleration engine featuring direct GPU-to-CXL FPGA loading with AMX optimization. Running ktransformer with unprecedented efficiency, ZettEngine delivers 10x cheaper inference while maintaining industry-leading performance metrics.

CXL 3.0 | FPGA Direct Load | AMX Optimized | ktransformer | GPU Offloading | OpenClaw Native

UNIFIED MEMORY TIERING/POOLING BASED ON PCIE/CXL

CPU+xPU with Unified Memory Tiering/Pooling can significantly enhance KV Cache performance in LLM inference, especially in high-concurrency, long-context workloads

Memory Domain	GPU HBM	CPU Memory	Tiered Memory
Latency	Lowest	Low	Comparable to CPU memory
Capacity	Limited	Moderate	Scalable (TB-level)
Cost per GB	High	Moderate	Lower than GPU DARAM
KVCache Suitability	Best for host cache	Good for mid-tier	Ideal for cold/pooled/shared
Scalability	Restricted	CPU Socked-bound	Fabric-wide

Scalability > Avoid vender lock-in, memory stranded, over provisioning

> Reduce TCO

> procurement and deployment > on demand, heterogeneous memory architecture

WHY THIS TALK NOW?

1 Hardware gap

Real multi-host CXL 3.0 hardware is not broadly available for researchers.

scarcity

2 Software gap

Applications still assume private memory + message passing, not pooled coherent memory.

compatibility

3 Scale gap

Single-host tools cannot expose the cross-host behaviors that matter at hyperscale.

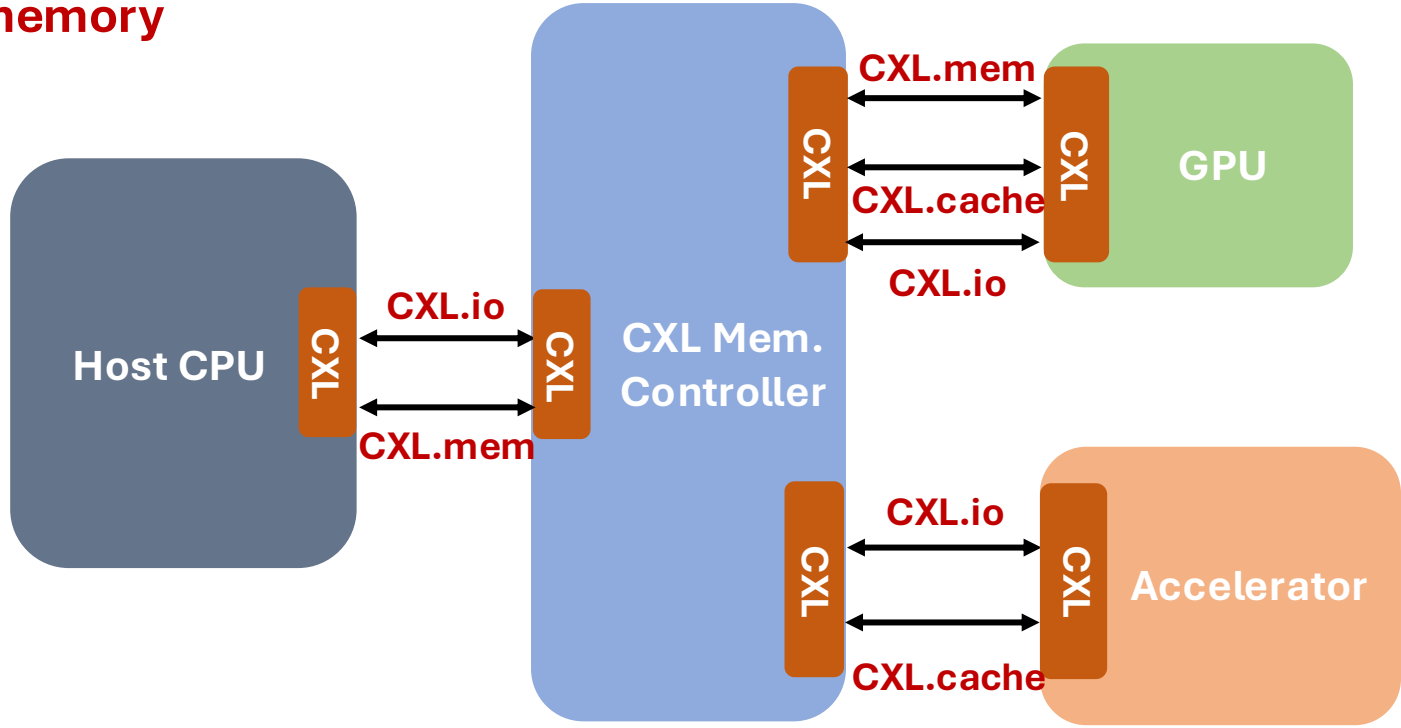
fidelity

-

INTRODUCTION TO CXL

Compute Express Link (CXL) builds on top of the PCIe5.0 interconnect by introducing **memory and coherency protocols**

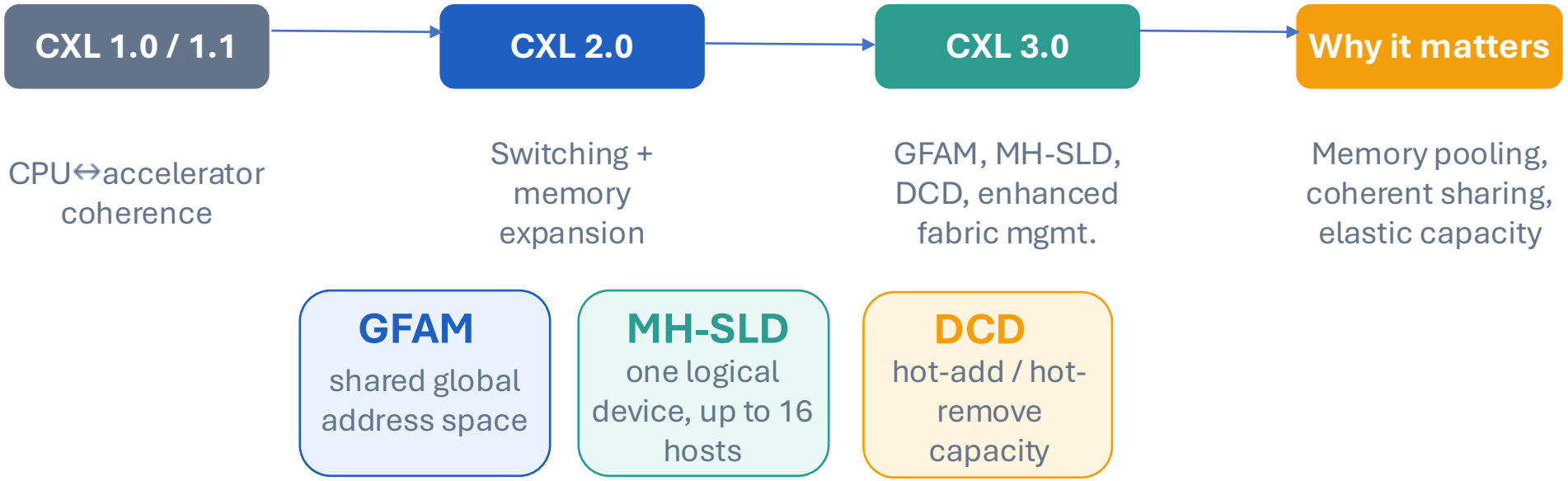
- **CXL.io**: required by devices; uses PCIe features
- **CXL.mem**: **allows host to directly access the memory** of other devices using load/store primitive
- **CXL.cache**: **allows devices to cache host's memory** using the MESI coherence protocol



Allows heterogeneous devices to achieve better memory capacity and bandwidth!

BRIEF HISTORY OF CXL

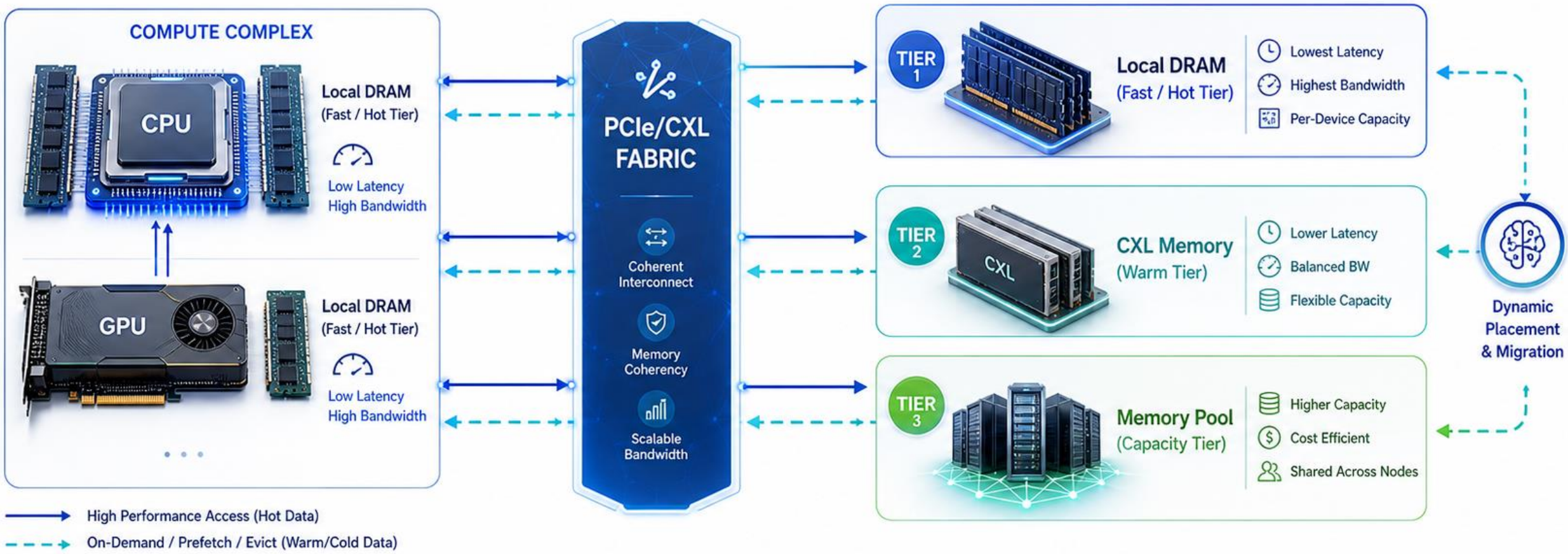
A compact evolution from coherent accelerator links to fabric-level memory sharing.



OCEAN is about making these CXL 3.0 capabilities testable today, at system scale.

Unified Memory Tiering/Pooling Based on PCIe/CXL

One Memory Space • Dynamic Tiering • Elastic Capacity • High Performance



UNIFIED MEMORY MANAGEMENT

- Global Address Space Across Devices & Pools
- Policy-Driven Tiering (Performance / Cost / Capacity)
- Real-Time Monitoring & Telemetry
- Workload-Aware Placement
- Elastic Scaling On Demand

Unified, Coherent, and Scalable Memory for Next-Generation Systems

CPU GPU RATIO EVOLUTION 1:8 ->4:1

Production multi-agent systems already show large execution amplification:

- Agents use about **4× more tokens** than chat interactions
- Multi-agent systems use about **15× more tokens** than chats
- For task allocation, Anthropic reports:
  - simple fact-finding: **1 agent, 3–10 tool calls**
  - direct comparison: **2–4 subagents, 10–15 calls each**
  - complex research: **10+ subagents**

CPU demand comes from six places:

1. **Orchestration:** planning loops, state machines, retries, branching
2. **Tool execution:** Python, search, DB queries, parsers, API wrappers
3. **Browser / OS interaction:** DOM extraction, screenshots, event handling, automation
4. **Memory & retrieval:** chunking, ranking, filtering, serialization
5. **Security boundaries:** sandboxing, isolation, policy checks, auditing
6. **Multi-agent coordination:** spawning workers, scheduling, merging results

WASM AS A CONTAINER SOLUTION

If agents are CPU-heavy, MVVM should be designed as an execution runtime, with careful pipelining, not just a model wrapper.

Securely

- isolate each agent / tool / browser session
- enforce least privilege
- make tool use auditable and policy-controlled

Efficiently

- keep tool execution close to CPU and data
- support warm pools, snapshot/resume, and parallel tool execution
- avoid starving GPUs while CPUs serialize the workflow

Everywhere

- run the same agent runtime across cloud, edge, and enterprise environments
- leverage ubiquitous CPU resources for orchestration and secure local execution

TERNARY MATMUL AS AN INFERENCE SOLUTION

Motivation
LLM inference is dominated by matrix multiplication, where both compute cost and memory bandwidth become bottlenecks. Reducing weight precision can directly lower data movement and simplify arithmetic.
Key Idea
Replace full-precision weights with ternary values:

$$W \in \{-, 1, 0 + 1\}$$

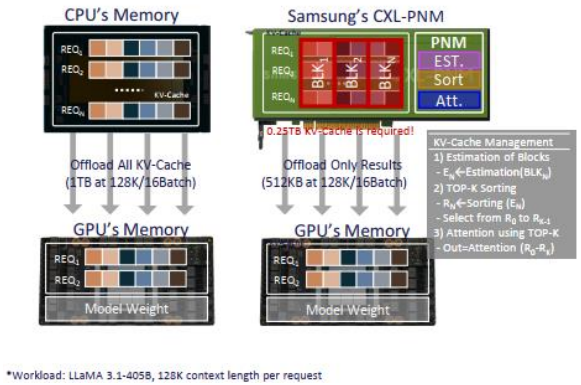
Then multiplication becomes much cheaper:

$$Y = XW \approx \alpha \cdot XW_t$$

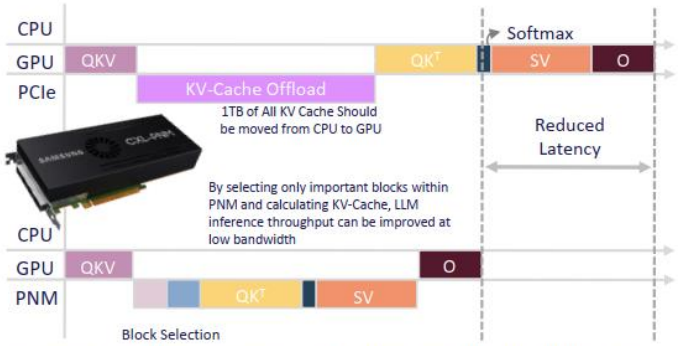
where $W_t$is the ternary weight matrix and $\alpha$is a scaling factor.

No expensive multiplications: operations become add, subtract, or skip.
Lower memory bandwidth: ternary weights require far fewer bits than FP16/INT8.
Sparsity benefit: zero weights can be skipped during inference.
Hardware-friendly: easier to implement with bit-packing and custom kernels.

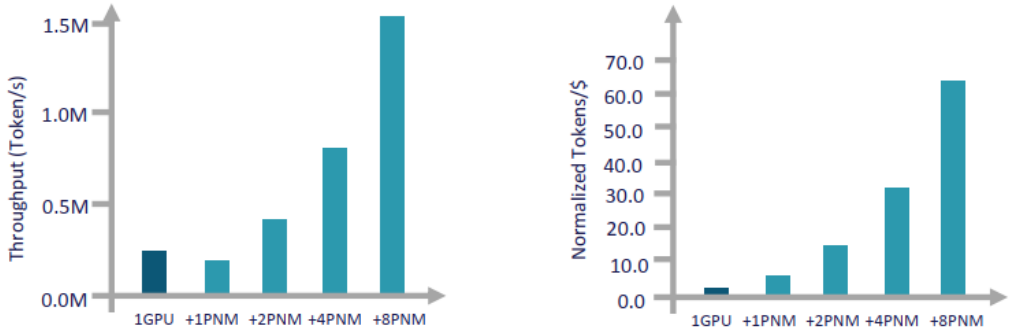
CXL-RPU(REMOTE PROCESSING UNIT) AN LLM-TURBO



*Workload: LLaMA 3.1-405B, 128K context length per request



< Computation of Attention Layers in CPU+GPU (Upper) and CPU+GPU+PNM (Lower) >



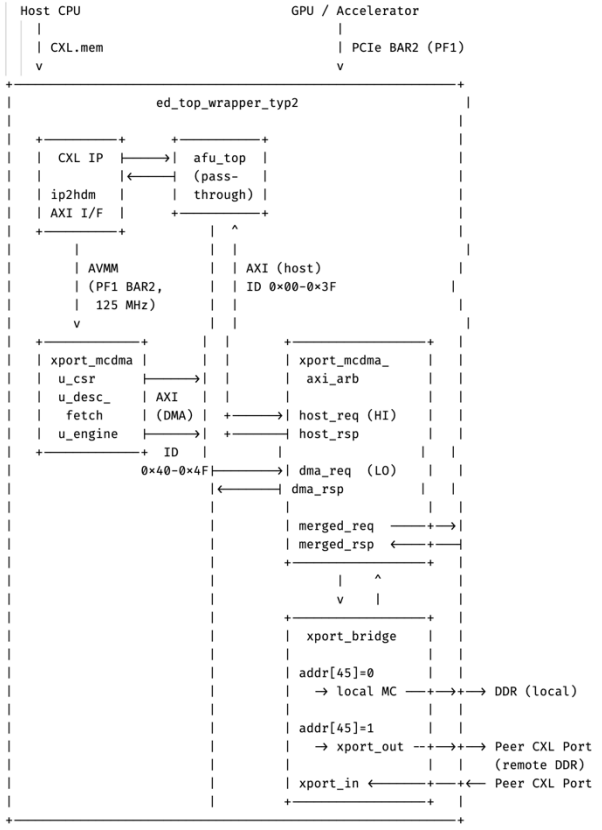
< End-to-End throughput/Efficiency comparison at LLaMa3.1-70B with Context length at 128K tokens >

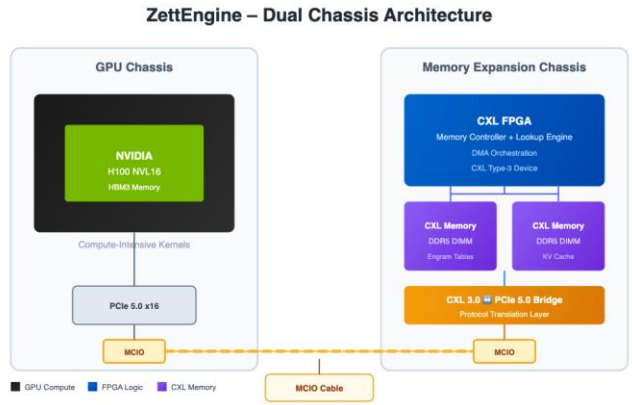
Efficient KV-Cache management running RPU for scalable LLM inference to long contexts

- CXL-RPU performs token block-based selection directly within CXL memory module, eliminating GPU's KV-Cache offload cost
- By storing KV-Cache in CXL memory and offloading selection to RPU, GPU memory pressure and support larger batch sizes
- With the integration of DPU-like(RPU) technology, future KV Cache operations can be processed with greater speed and efficiency

CXL-RPU(REMOTE PROCESSING UNIT) A CONNECTOR

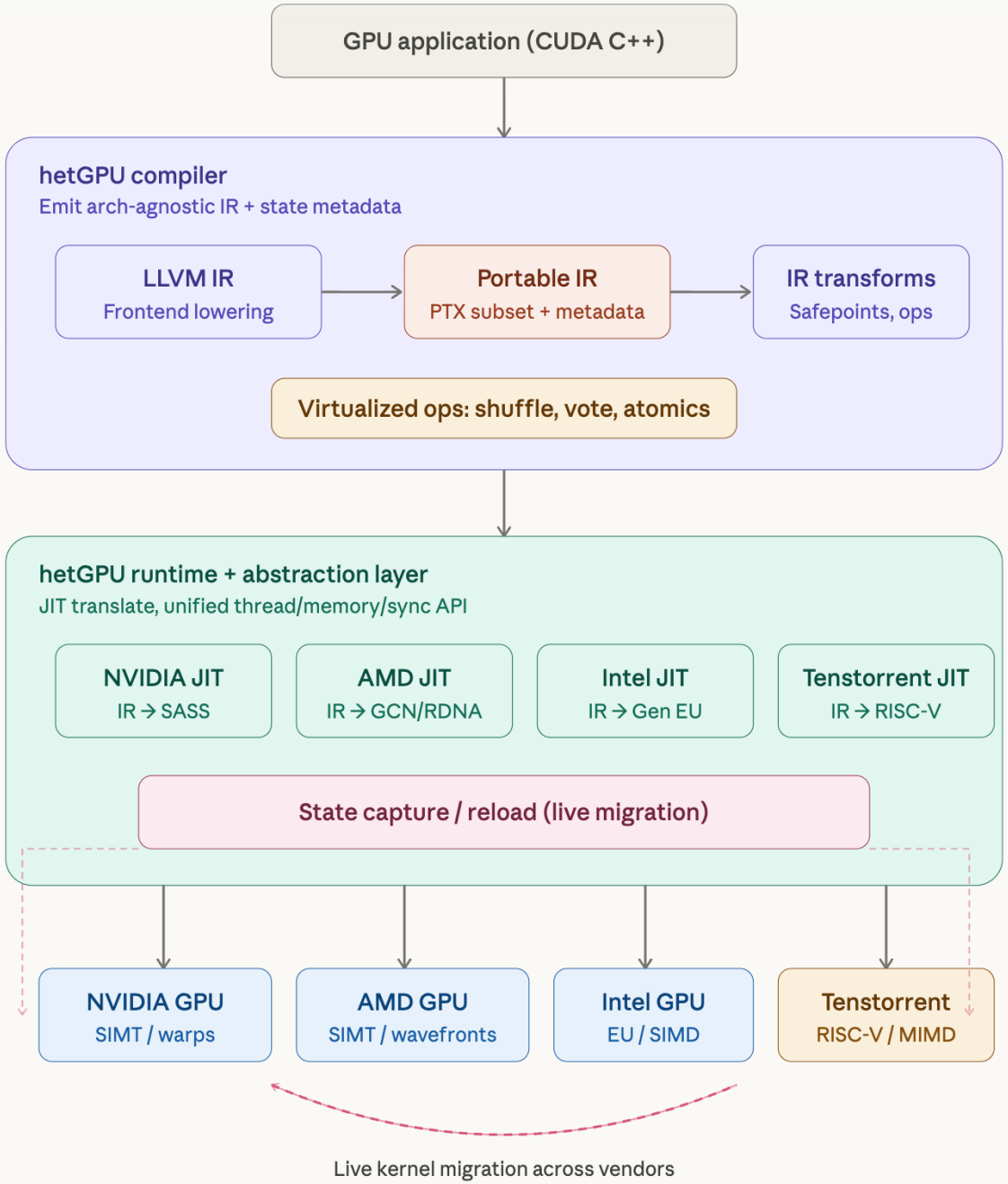
- Could act as both device and host
- RISC-V Soft Core for Data Prefetching from PCIe to CXL
- Engram-style conditional memory introduces a lookup--compute separation, where large memory tables are accessed via indexed lookup with constant-time complexity
- Disaggregating HBM to CXL Memory Just like Storage Service brought by Nvidia.





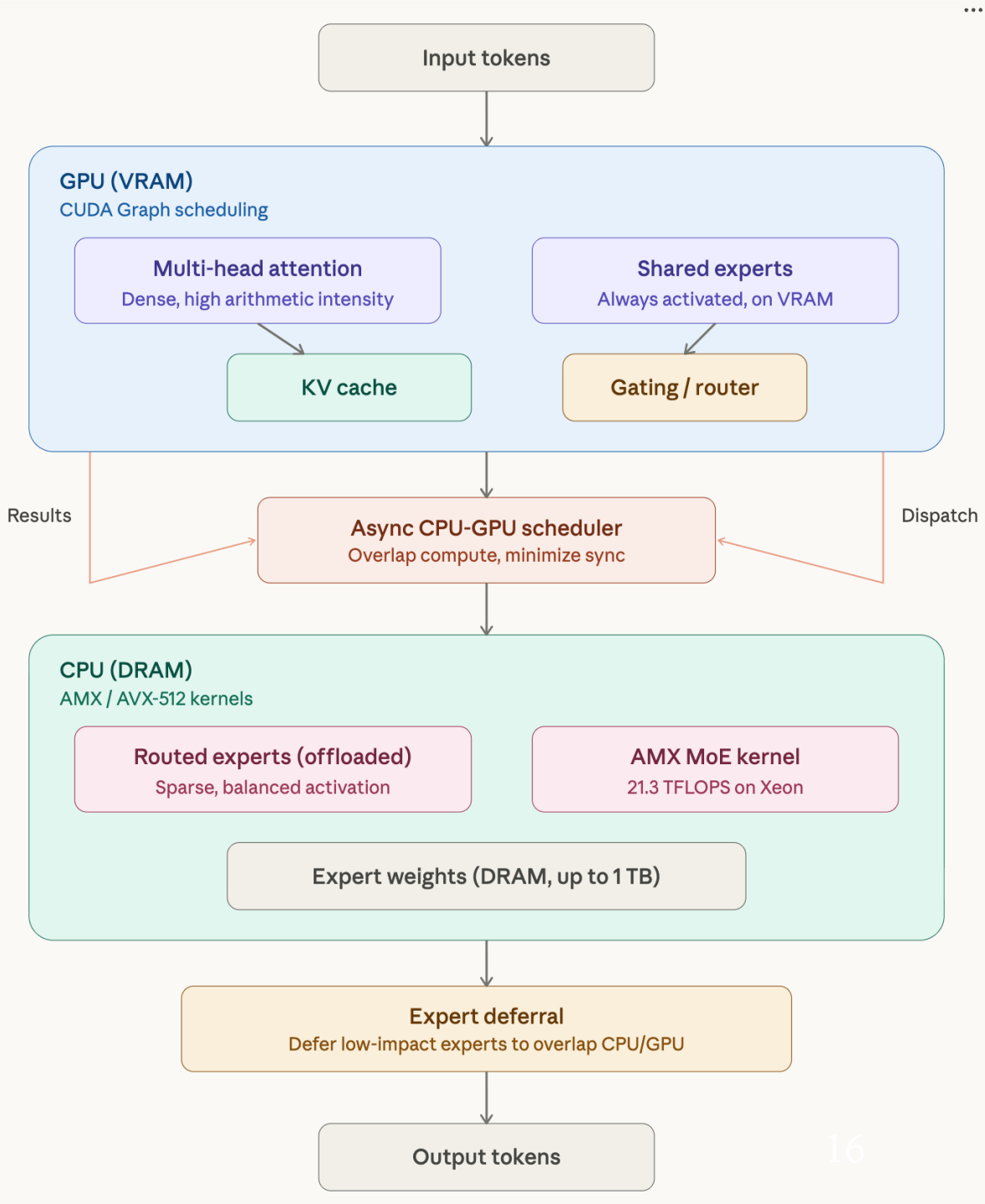
THE SOFTWARE STACK

- CXL-RPU acts as a connector layer between GPUs, CPUs, system memory, and remote processing resources.
- The compiler stack lowers CUDA-style GPU applications into a portable intermediate representation before mapping them to different hardware backends.
- Vendor-specific JIT backends support NVIDIA, AMD, Intel, and Tenstorrent accelerators while preserving a unified programming model.
- Runtime support enables state capture, live migration, SIMT-to-MIMD bridging, and memory-model unification across heterogeneous devices.
- For large-model inference, latency-sensitive attention and KV-cache operations remain on GPU VRAM.
- Sparse expert computation and large expert weights are offloaded to CPU DRAM, where AMX/AVX-512 kernels accelerate MoE execution.



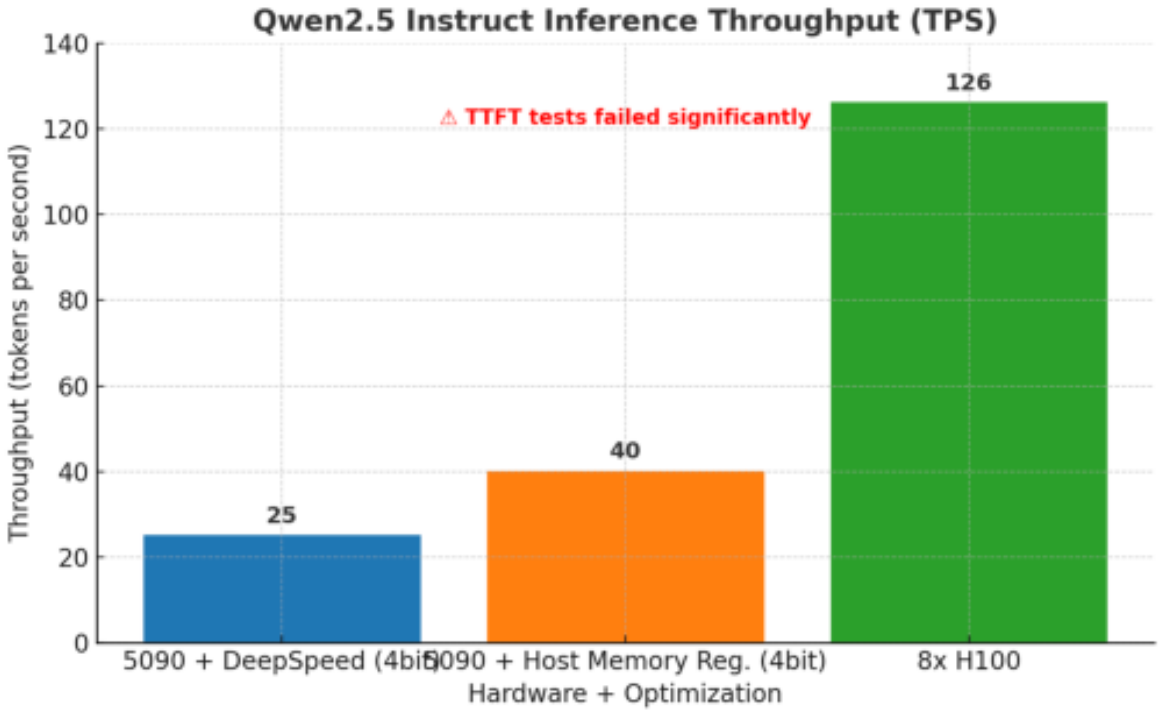
THE SOFTWARE STACK

- An asynchronous CPU-GPU scheduler overlaps compute, dispatch, and result collection to reduce synchronization overhead.
- Expert deferral allows low-impact expert work to spill over to CPU/GPU resources without blocking the critical inference path.
- Overall, the design positions CXL-RPU as a vendor-neutral bridge for scaling memory capacity and compute flexibility beyond a single local GPU.

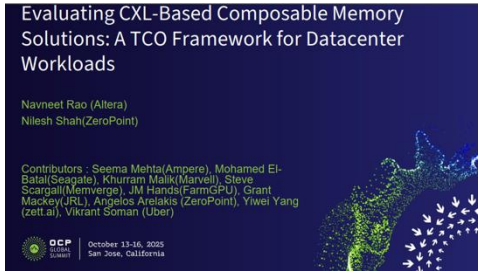


CXL GPU DIRECT-MEMORY FOR LOWER DC-TCO

Metric	8x H100 GPUs	5090+CXL Solution	Cloud Solution
Hardware Capability	✅ Just fit run 671B (640GB < 700GB needed)	✅ Can run quantized version	✅ Full model support
Actual Performance	40 TPS	6-8 TPS (needs optimization to 40)	40+ TPS
Initial Investment	$400,000	$6,000	$0
Power Cost (3yr)	$62,100	$15,000	Included
Hosting/Colocation (3yr)	$108,000	$36,000	Included
Maintenance/Support (3yr)	$60,000	$10,000	Included
Total TCO (3yr)	**$630,100**	**$67,000**	**$394,200**
Cost per 1K Tokens	$0.002	$0.0001	$0.003



1. Host CPU that supports CXL p2p like latest thread ripper
2. Nvidia GPU like 5090 that direct load store to memory pool
3. Altera CXL FPGA for compression and translating PCIe to CXL p2p
4. CXL memory pool with up to 1TB
5. Deepseek 671B 25TPS can be comparable to 8 cards H100



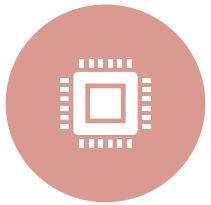
OCP work

MONETIZATION

INFERENCE-AS-A-SERVICE (IAAS) WITH TCO ARBITRAGE.

FPGA IP LICENSING + REFERENCE DESIGN.

CXL MEMORY FABRIC SOFTWARE STACK.



TURNKEY APPLIANCE FOR ON-PREM LLM INFERENCE.

DEVELOPER TOOLS AND BENCHMARKING SERVICES.

CXL-NATIVE CMX ALTERNATIVE FOR NEOCLOUDS.

CLOSE

TAKEAWAYS

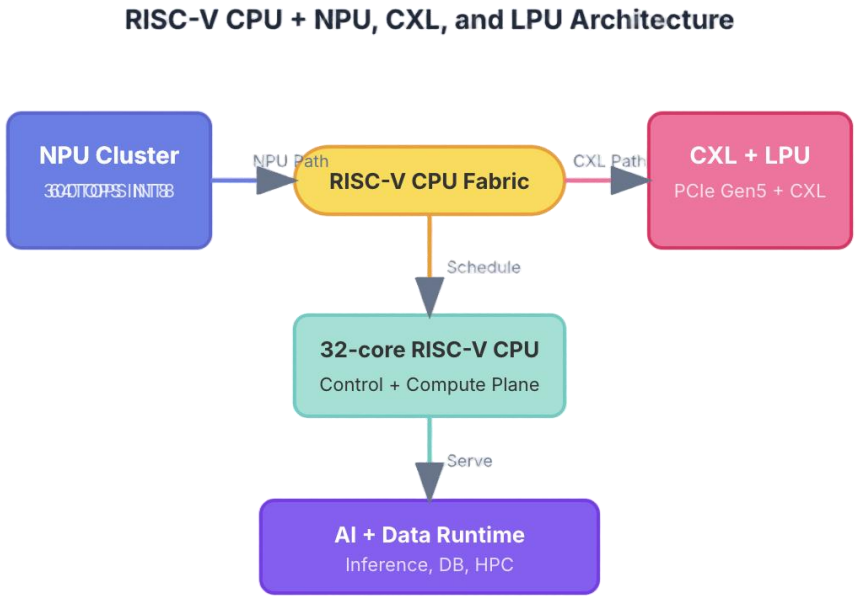
1	**CXL 3.0 is most interesting when multiple hosts share memory coherently.**	That is exactly where commodity hardware is currently least accessible.
2	**CXL fills that gap with full-system emulation.**	The guest OS and applications observe timing and correctness effects, not just traces.
3	**The data matters.**	The result shows an 11× coherence gain, a ~7% policy spread, and hardware-calibrated timing behavior.
4	**The tutorial's goal**	Attendees should leave knowing what to run next and where to contribute.

-

ZETT.AI

zett.ai

- zett.ai organizes the platform around two building blocks: a server-class RISC-V CPU + programmable NPU compute base, and a CXL/LPU expansion layer for memory, acceleration, storage, and high-speed I/O.
- zett.ai is aimed at concrete infrastructure deployments where GPU memory, local inference, and expandable memory capacity directly affect cost, density, and service latency.



https://zettai.us/

vickie@zettai.us

Questions?